AIM PROTOCOL

Agri • Identity • Microfinance

Version 2.2 — Release Notes & Technical Reference

Built by Amjad Kamara
Founder, Aadios Systems (SL) Ltd. • *Sierra Leone*
Post-Hackathon Development — June 2026

Live: https://aim-protocol.vercel.app/

Status	**Live on Solana Devnet**
Frontend Repo	github.com/amjadkamara/aim-protocol
Smart Contract Repo	github.com/amjadkamara/aim-program
Program ID	AhHHJTu5vodDYE2yLNet2bE6jad9F3xSfbLQdUmykKqB
Admin Wallet	Cz3GvsRaBsuAHoRiJd5sV6ZTAkE8TsFJAyuYWEtV7Qu2
Context	Post-Hackathon — Solana Frontier 2026

1. Overview

V2.1 delivered a production-ready loan lifecycle — a farmer could register, borrow, repay, and re-borrow with full on-chain transparency. V2.2 delivers the first administrative layer: a wallet-gated dashboard that allows the platform operator to view all registered farmers and their loan status directly from Solana devnet in real time.

This is the foundation for multi-lender onboarding in V2.3. When an NGO or cooperative connects their wallet to AIM Protocol, they will see a version of this dashboard scoped to their own farmers and loans. V2.2 builds and proves that infrastructure with a single operator wallet.

2. What V2.2 Built

2.1 Admin Dashboard

File: src/AdminDashboard.jsx (NEW)

A dedicated screen accessible only to the platform operator wallet. Fetches all FarmerAccount PDAs on-chain using Anchor's program.account.farmerAccount.all() and enriches each record with its corresponding LoanAccount PDA data.

Aggregate Statistics

Stat Card	Source
Total Farmers	Count of all FarmerAccount PDAs returned by program.account.farmerAccount.all()
Active Loans	Count of LoanAccounts where status === active
Overdue	Count of LoanAccounts where status === overdue
Repaid	Count of LoanAccounts where status === repaid
Repayment Rate	repaid / totalFarmers x 100 — calculated live from on-chain data
Total SOL Issued	Sum of all LoanAccount amounts in lamports, converted to SOL

Farmer Table

- Columns: Farmer Name, District, Farm Size, Wallet Address (shortened), Crop Type, Loan Amount (SOL), Loan Status
- Sortable columns — Farmer name, Crop type, Loan amount, Status
- Search by farmer name, wallet address, or crop type
- Filter buttons — All / Active / Overdue / Repaid / No Loan
- Solana Explorer link per farmer PDA — opens on-chain account directly
- Refresh button — re-fetches all on-chain data on demand
- Overdue rows highlighted in red for immediate operator visibility
- Mobile responsive layout matching V2.1 design system

2.2 Wallet-Gated Access Control

File: src/AdminDashboard.jsx — ADMIN_WALLETS constant

Three access states are enforced at the component level. No backend or server required — all access control is derived from the connected wallet public key compared against the hardcoded ADMIN_WALLETS array:

State	Condition	Screen Shown
Wallet Required	No wallet connected	Shield icon with prompt to connect admin wallet and back to home navigation
Access Denied	Non-admin wallet connected	Red shield icon, Access Denied heading, full wallet address displayed
Dashboard	Admin wallet connected	Full farmer table with stat cards, search, filter, and sort

Security Note:

The ADMIN_WALLETS constant stores the platform operator's Solana public key. Public keys are safe to commit to source control — they are visible on-chain and contain no sensitive information. This is categorically different from private keys and seed phrases, which must never be committed to any repository.

2.3 getLoanStatus() Defensive Helper

File: src/AdminDashboard.jsx

A defensive helper function wraps all Object.keys(loan.status) calls to handle null, undefined, or malformed status objects from legacy on-chain accounts created before V2.1.

Bug Identified During V2.2 Testing:

On initial load, the admin dashboard crashed immediately with a TypeError. The cause: legacy FarmerAccount PDAs created before V2.1 had loan status objects in an unexpected shape. Object.keys() on a null or malformed status object threw an uncaught exception that unmounted the entire dashboard component.

Resolution: getLoanStatus() wraps the call in try/catch and returns null for any account that cannot be parsed. The dashboard renders cleanly for all account shapes.

2.4 App Navigation Update

File: src/App.jsx (UPDATED)

- AdminDashboard imported and wired to the page === 'admin' route
- Admin button added to the home screen hero button group for connected wallets
- onBack prop routes the admin dashboard back to home, consistent with all other screens

2.5 Screenshot Documentation Library

Folder: docs/screenshots/

16 folders established covering the complete user journey from homepage through admin dashboard. Screenshots captured for both the V2.1 farmer journey and V2.2 admin security states.

Folders	Contents
01 — 10	Full farmer journey — homepage, wallet connect, Farmer ID registration, microloan request, repayment, loan history, dashboard states
11 — admin-overview	Admin dashboard overview, Wallet Required state, Access Denied state
12 — farmers-table	Farmer table with loan amounts and status badges
13 — loan-monitoring	Loan monitoring view and filter states
15 — farmer-profile-admin	Reserved for V2.3 farmer profile drill-down
16 — system-architecture	Reserved for architecture diagrams

3. Technical Notes

3.1 On-Chain Data Fetch Strategy

V2.2 uses getProgramAccounts via Anchor's account namespace (program.account.farmerAccount.all()) rather than manual RPC filter calls. This method automatically applies the correct account discriminator filter for FarmerAccount structs, ensuring only valid farmer accounts are returned regardless of other program accounts present on devnet.

For each farmer record returned, a secondary fetch of the corresponding LoanAccount PDA is performed using getLoanPDA(farmerData.owner). This secondary fetch is wrapped in a try/catch — farmers with no loan history display cleanly with a No Loan badge rather than crashing the render loop.

```
Fetch pattern:
const allFarmers = await program.account.farmerAccount.all()
for each farmer:
  try { loan = await program.account.loanAccount.fetch(getLoanPDA(farmer.owner)) }
```

```
catch { loan = null }  // no loan — render cleanly
```

3.2 Test Data Condition

All V2.1 test farmers were registered from a single wallet. The admin dashboard correctly reflects this — loan amounts appear identical across multiple rows because they resolve to the same loan PDA. This is a test data condition, not a bug.

Independent Wallet Verification:

Farmer Binta Marie Winner registered from wallet 2TDJWeL4wzGYvSEddbgnTuJJuUbjpreS7zvEPPrpkjb1 on 02 June 2026 — separate loan PDA, unique loan amount, and distinct status confirmed. Appears as an independent row in the admin dashboard with no overlap.

This confirms the multi-wallet architecture works correctly. V2.3 will start with a clean devnet state using separate wallets for Admin, Farmer 1, Farmer 2, and Lender.

4. File Changelog

File	Status	What Changed
src/AdminDashboard.jsx	New	Full admin dashboard — wallet-gated access control, farmer table, aggregate stat cards, search, filter, sort, getLoanStatus defensive helper, Solana Explorer links, refresh button
src/App.jsx	Updated	AdminDashboard import, admin page route added, Admin button added to home hero button group
docs/screenshots/	New	16-folder screenshot documentation library covering V2.1 farmer journey and V2.2 admin states

5. V2.2 Verification Checklist

- ✓ Admin wallet connected — full dashboard loads with all farmers from devnet
- ✓ Wallet disconnected — Wallet Required screen displayed correctly
- ✓ Different wallet connected — Access Denied screen with full wallet address shown
- ✓ Farmer table loads and displays all registered farmers correctly
- ✓ Search filters farmers by name, wallet address, and crop type
- ✓ Status filter buttons (All / Active / Overdue / Repaid / No Loan) work correctly

- ✓ Column sorting works for name, crop, loan amount, and status
- ✓ Solana Explorer links open correct farmer PDA accounts on devnet
- ✓ Overdue rows highlighted in red
- ✓ Refresh button re-fetches live on-chain data
- ✓ Independent wallet tested — Binta Marie Winner appears as separate row
- ✓ getLoanStatus() crash fix — no TypeError on legacy account shapes
- ✓ Mobile responsive — dashboard usable on smaller viewports
- ✓ Deployed to Vercel — live at aim-protocol.vercel.app
- ✓ Committed to GitHub — github.com/amjadkamara/aim-protocol

6. V2.3 Roadmap

Smart Contract

- register_lender instruction — LenderAccount PDA per lender wallet
- Loans tagged with lender pubkey for multi-lender portfolio queries
- Credit score derived from repayment history — stored in FarmerAccount
- Loan counter per farmer for proper history indexing across lenders
- Admin role enforcement — admin wallet locked out of Farmer and Lender roles

Frontend

- Authenticated home state — personalised welcome greeting for registered farmers
- Lender onboarding flow — multi-step registration form for organisations
- Per-lender dashboard — lenders see only their own farmers and loans
- Farmer-facing loan marketplace — browse available lenders and their terms
- Public farmer profile lookup by wallet address

Business

- Pilot with one farming cooperative in Sierra Leone — 10 farmers on devnet
- Superteam Africa community — introduce AIM Protocol to Solana Africa builders
- Grant applications — Solana Foundation African Ecosystem, IFAD AgriFinance
- Aadios Systems company profile and pitch deck

AIM Protocol V2.2 — Admin infrastructure complete. Multi-lender foundation ready for V2.3.

Amjad Kamara | Aadios Systems (SL) Ltd. | https://aim-protocol.vercel.app
Built from Sierra Leone SL